

Python Simulation and Animation of Planetary Motion Around the Sun

Author: Matthew Birnhak

Introduction

I created a simulation and animation in python, utilizing the Beeman integration scheme, to demonstrate the movement of the inner 5 planets around the sun. The simulation's initialization parameters are read in from a 'parameters-solar.txt' input file, in which the other planets can be added by uncommenting them. After finishing the initial Beeman simulation and animation, I tackled 3 different experiments. First, I modified my original code to determine the orbital period of each planet, to see how close the simulation's orbital period was to the actual orbital period for each planet. For the second experiment I created a separate simulation and animation for the Direct Euler integration scheme. The aim of this experiment is to compare the energy conservation of the Beeman scheme to that of the Direct Euler, and to discuss the differences of the two integration methods. Finally, the last experiment I chose to do was experiment 4 in which I determined when the planets are all aligned with each other within some threshold angle (e.g. 5°). The aim was to calculate the planet's angles from the positive x-axis (while also accounting for the sun's movement in the y direction), and to print out when they are all aligned within the threshold value. This experiment had more verifiable results when run with 4 planets, because as the amount of planets increased the frequency with which they were all aligned decreased drastically.

Methods

My program is sectioned into 4 different python files to make it more manageable to use and understand. I have taken some inspiration from the provided templates, however, I do not directly use any of them. I use the concept of the 'Planet' class and the method for processing the input file, but I have modified both to fit my needs. I have created one file labeled 'planet.py' which is a small file containing a planet class. This class is created with only an initialization function to store all of the data for the sun and each of the planets.

The second file is labeled 'beeman.py' and contains a class called 'SolarSystem'. As the filename suggests, this class runs the Beeman integration simulation. This class imports the 'Planet' class and stores all of the planetary information in an array of 'Planet' objects. This allows me to easily add and remove planets without having to change the structure of my code. The 'SolarSystem' class contains methods for updating each celestial body's acceleration, position, and velocity based on the Beeman integration scheme. I loop over all celestial bodies, which again allows for the easy addition or removal of planets. Furthermore, the class has a method for simulating one time step, which I loop over in the 'simulation' function for the number of timesteps given in the input file. The class also contains a method for calculating the total energy, which sums up the result of calling a kinetic energy function and a potential energy function. By separating the two separate energy calculations I can simplify what would be a complex function for calculating total energy into two simple function calls. I implement two

more functions to complete experiment 1 - calculating the orbital period of each planet. First, I create a 'new_year' function which returns true when the position of the planet object (passed in as a parameter) has passed from below to above the positive x-axis. The second function, 'orbital_period', iterates over all of the planets and calls the 'new_year' function to determine when to print a new orbital period. I do this to easily allow me to call this method at each timestep and successfully print out each planet's new year. Finally, I implement a method labeled 'alignment' in order to complete experiment 4 - planetary alignment. This function, although quite short, is a little more complicated than the other functions created. The function takes a threshold angle, determines the angles of all the planets in the system relative to the positive x-axis, and calculates if the difference between all of the planets is below the threshold. However, there are a few subtleties within the code. First, I calculate the angle of each planet based on its x coordinate for position, however, I use the y coordinate minus the y component of the sun's position. This was done to make sure the angles were being taken relative to the line parallel to the positive x-axis passing through the center of the sun. The second complexity within this method is the complex list comprehension done in line 170. This utilizes python's advanced feature to loop through all of the planets, calculate their angles, modulo their angles by 2 times pi (to convert into the range $(0, 2\pi)$), and store all of these angles in an array in one line. While this is a complex line of code, I have done it to reduce the overall length and complexity of my 'alignment' method. I also design this method to easily call it at every timestep, just as with the orbital period function, and print out when alignment is reached.

The third file is labeled 'euler.py' and it contains one class, 'SolarEuler'. This python class inherits the 'Solar_system' class above, and overwrites the methods for updating each celestial body's acceleration, position, and velocity based on the Direct Euler integration scheme. I also overwrite the methods for simulating one time step and simulation as the Direct Euler scheme has a slightly different order for calculation. I implement the structure of this file in this way to limit the duplication of code needed to create the class for the Euler method. I know that it is following the same overall structure of the 'beeman.py' file, but with the obvious exception of using a different integration scheme. Therefore, by using the object-oriented programming technique of inheritance I do not need to duplicate the code for calculating energy.

The final file in my program is labeled 'run.py', which is used to run the code and see the results of the different experiments. I originally was implementing all 4 files in this single file, however, it was cluttered and difficult to read. Therefore, I took appropriate sections out and broke them into manageable parts. I could do this seamlessly because of the object-oriented style of programming I used to structure the entire project. I can simply cut out entire classes as needed and put them in separate files, which can then be imported for use. My final file contains an 'Animation' class, an 'energy_comparison' function, and a 'main' function. The 'Animation' class simply runs the animation for the integration method desired, which allows you to see the motion of the planets around the sun graphically. I have implemented a parameter labeled 'method' that I pass in to determine which animation to run. This allows for the animation to run as desired by entering user input, rather than having to go in and change code. The 'energy_comparison' function, which is not in any class, is used for experiment 2 - energy comparison between the different integration methods. I designed this method outside of any class because it did not make sense to fit it into any of the classes defined as it uses both the Beeman and Euler methods. Furthermore, it is too large of a code to put into the main without cluttering it, so I decided to make it a method that can be called in the main method with one

line. This function runs the simulation for both Beeman and Euler and plots the comparison of their energies. The one interesting component here is my use of the ‘deepcopy()’ function. I use this to make a copy of the ‘Planet’ objects array passed in, so that the two different simulations are not editing the same bodies. Finally, in my main method I read and process the input file, and call the desired functions. I implement the main to ask for a few very simple inputs to make the file more easily run. The user supplies a letter corresponding to either a simulation, animation, or energy comparison, and, if energy comparison is not chosen, a letter for the integration method to use. This allows the user to easily run a simulation or animation for either interaction method, or an energy comparison, with no need to change the code.

Lastly, the initial conditions I use are a timestep of 0.003 earth years, 100,000 iterations, and the 5 inner planets. I use a timestep of 0.003 as it is close to that of a single earth day (1/365), and I run for 100,000 iterations because that allows the simulation to run for 300 years. This is a good time step and number of iterations for most of the experiments. The only exception is that when it comes to the planetary alignment method it is best to run it with 4 planets and for more timesteps. Alignment of all 5 planets happens very infrequently, so it requires running for a very long time. While alignment of the inner 4 also happens somewhat infrequently, it happens much more often than with 5. Another solution is to increase the threshold angle from 5 to 10 degrees.

Results

Experiment 1: Orbital Periods

Table 1. Observed python simulation and the actual orbital periods of the inner 5 planets

Planet	Orbital Period (Earth Years) - Simulation	Orbital Period (Earth Years) - Actual	Percent Error (%)
Mercury	0.24	0.24	0.00
Venus	0.61	0.62	1.61
Earth	1.00	1.00	0.00
Mars	1.88	1.88	0.00
Jupiter	11.82	11.86	0.34

Experiment 2: Energy Conservation and Alternative Integration Methods

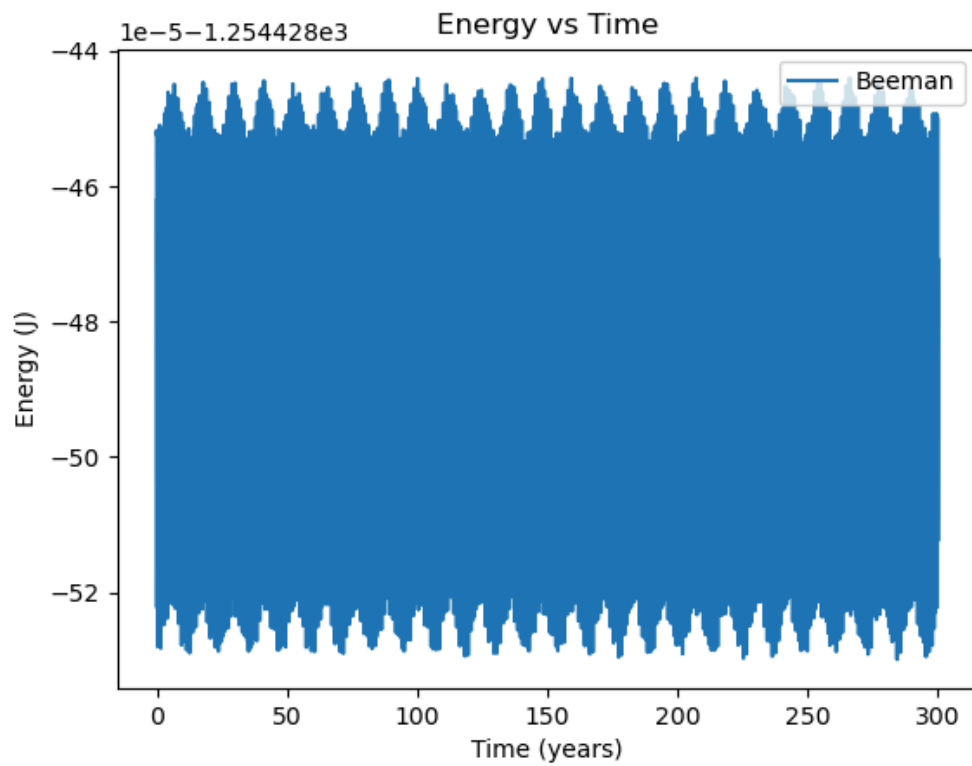


Figure 1. Total energy of the python Beeman integration simulation over 300 years with a time step of 0.003 earth years

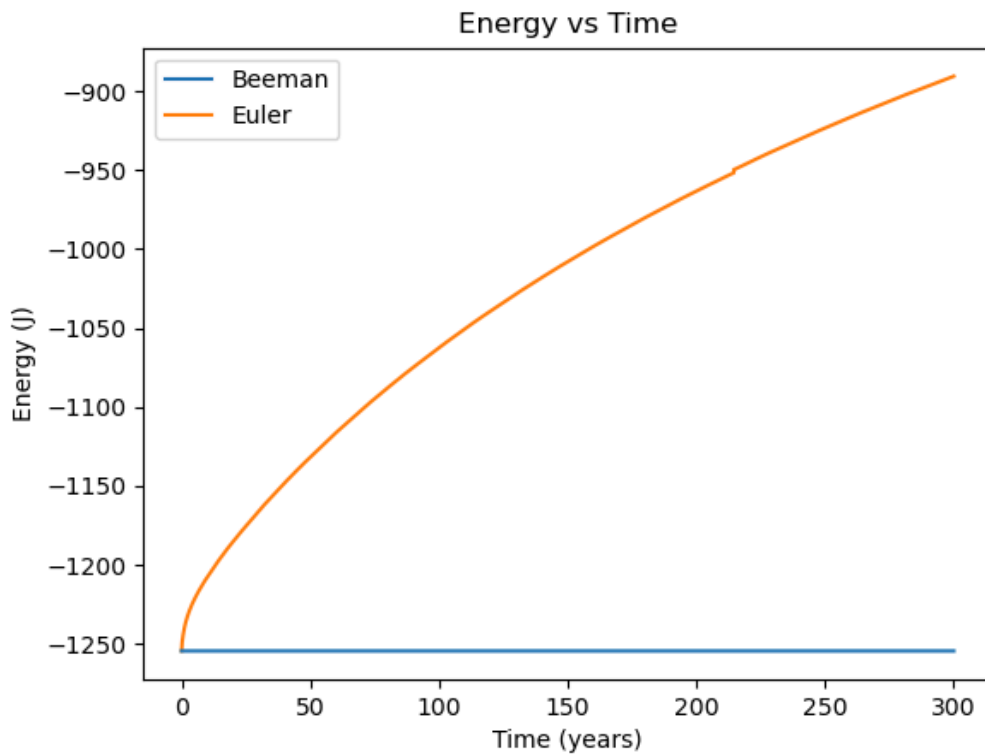


Figure 2. Total energy of the python Beeman integration simulation compared with the Direct Euler integration simulation over 300 years with a time step of 0.003 earth years

Experiment 4: Planetary Alignment

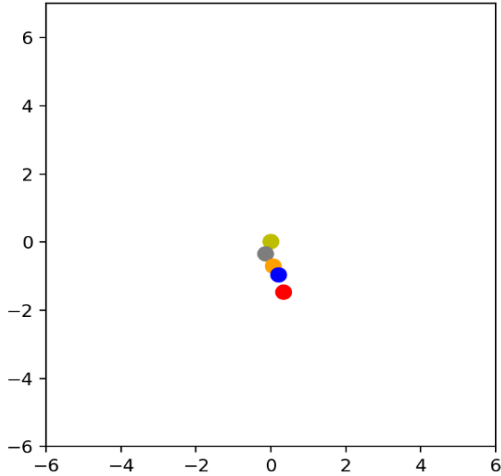


Figure 3. Freeze frame of python animation during planetary alignment (within 5°)

Table 2. Number and frequency of alignments with 4 planets and 5 planets over 3,000 years with a timestep of 0.003 earth years and threshold of 5° or 10°

	4 Planets: 5° Threshold	4 Planets: 10° Threshold	5 Planets: 5° Threshold	5 Planets: 10° Threshold
# of alignments	7	26	0	3
Average time between alignment (years)	492.71	113.70	0	1291.40

excludes alignments in consecutive timesteps from calculations

Discussion

Experiment 1 demonstrates that the python simulation based on the Beeman integration scheme maintains an accurate representation of the orbital periods for the inner 5 planets. As seen in table 1, the orbital periods of 3 of the planets is 0, and the highest percent error is below 2%. These results were found by calculating the current time of the simulation and dividing it by how many years that planet has already simulated. These calculations came out to orbital periods equal to or very close to actual values, indicating that the simulation and calculations are accurate. However, it is likely when adding in the outer planets, these values will begin to deviate slightly more from their actual values due to the orbital radius increasing.

Experiment 2 shows an interesting trend in energy for the Direct Euler scheme compared to the Beeman. First, the energy of the Beeman scheme, as seen in figure 1, is maintained well throughout the entire simulation. This is a key component of the Beeman integration method, as

it is beneficial because it conserves energy, and figure 1 shows that my simulation follows this. However, as seen in figure 2, the Direct Euler scheme has a very large logarithmic-like curve in its energy. At first this result seemed quite alarming, however, after discussion with peers it appears this is the trend seen when implementing Direct Euler. After discussion with peers, it seems that when using Euler-Cromer, rather than direct Euler, it is possible to get oscillations that diverge in energy over time. Furthermore, the results of figure 2 show that similar to figure 1 Beeman conserves energy over time, however, Direct Euler does not conserve energy well at all. This is indicative of the two different methods overall. Direct Euler, while it does not conserve energy well, is a much simpler and easier to implement method. However, while Beeman is more complex and difficult to implement, it is a more accurate model that conserves energy.

Experiment 4 illustrates how often planetary alignment occurs between both the 4 and 5 innermost planets. An example of this from my animation can be seen in figure 3. I utilized a threshold of 5° and 10° so that there was some amount of alignment within the 5 planet simulation. Table 2 shows, as expected, that with a smaller number of planets there is a much higher frequency of alignment. This is accurate with real life and makes sense because as the number of planets increases, the number of orbits that have to line up increases. As a result, the probability of them all aligning will decrease. Furthermore, as seen in table 2, as the threshold angle increases, the frequency of alignment also increases (average time between alignment decreases). This also makes sense because if the threshold range that the angles have to be in is a larger region, then there is a greater likelihood that all planets will align within that threshold. An interesting pattern that was seen when calculating the average time between alignments was that the time between consecutive alignments seemed to change on average by a certain constant factor. This is likely accurate as all of the planets are being simulated with a certain integration method, and therefore, there is most likely a mathematical pattern that can be calculated to determine the frequency of alignment. Determining this was beyond the scope of the project and was not possible given the time constraints. Lastly, it is difficult to confirm whether these results are entirely accurate because checking all of my calculations over hundreds of thousands of iterations is simply not possible. However, given the semi-consistent pattern of finding alignments and the number of alignments changing with proportion to number of planets and threshold angle, it seems as though my results are accurate.

Conclusion

In this project I aim to simulate and animate planetary motion around the sun with the Beeman integration scheme. The final program successfully illustrates the motion of the planets around the sun accurately to the Beeman method. I carry out three different experiments to compare the integration method and illustrate its validity. I calculate orbital periods, I compare total energy over time to the Direct Euler method, and I determine planetary alignment based on a threshold angle. All three experiments are successfully completed and demonstrate relatively accurate orbital periods, planetary alignment frequency, and energy conservation. I have learned much about the modeling of planetary motion through different integration schemes, and how to utilize python to model real world problems. Unfortunately with the time constraint I am not able to attempt the Euler-Cromer method for comparison nor try and calculate the frequency with which the planets should align based on the Beeman method. However, I would like to investigate these eventually. Lastly, with the time constraint I was also unable to attempt sending a satellite from

Earth to Mars in my simulation, but I think this would teach me a lot and I would love to implement it in the future.